

COMPUTER NETWORKS LA-7

CUSTOM LIGHTWEIGHT DISCOVERY PROTOCOL (CLDP) SPECIFICATION

TUHIN MONDAL
22CS10087

Computer Networks LA-7 Documentation

Custom Lightweight Discovery Protocol (CLDP) Specification

Name - Tuhin Mondal

Roll - 22CS10087

Introduction

The Custom Lightweight Discovery Protocol (CLDP) is a purpose-built protocol aimed at closed network environments where traditional network discovery methods might not be ideal or secure. In CLDP, every node on the network has the ability to announce its presence, ensuring that all participants are aware of each other immediately after connecting. This announcement is essential for environments where nodes need to discover and authenticate peers before engaging in further communication.

CLDP uses raw sockets to bypass typical operating system network stack abstractions. This means that the protocol manually constructs and interprets its packets directly at the IP level. By doing so, it creates custom packets that include both a manually crafted IP header and a specialized payload conforming to the CLDP format. A key aspect of these packets is the use of a custom protocol number, ensuring that the operating system and any monitoring tools can easily distinguish CLDP traffic from regular IP traffic.

Protocol Details

- 1) Protocol Number: 253
- 2) Used directly over IP Layer
- 3) Metadata Supported:
 - a) **Hostname** – The unique name assigned to a system for network identification.
 - b) **System Time** – The current date and time maintained by the system's clock.
 - c) **CPU Load** – The percentage of processing power currently being utilized by the system.
 - d) **Uptime** – The total time the system has been running since its last reboot.
 - e) **Memory Usage** – The amount of RAM currently in use versus the total available memory.

Packet Structure:

Each CLDP Packet consists of a custom header and an optional payload

Header Format (8 bytes):

uint8_t packet_type:

This is an 8-bit unsigned integer used to indicate the type of the packet. For example, specific numeric values can denote different packet types, such as HELLO, QUERY, or RESPONSE in the protocol.

uint8_t packet_length:

Another 8-bit unsigned integer, typically used to specify the length of the payload data (or overall header length for certain operations). This helps in determining how much data to read following this header.

uint16_t transaction_id:

A 16-bit unsigned integer that uniquely identifies a given transaction. This allows matching of requests with the corresponding responses and facilitates tracking of communication sessions.

uint32_t reserved:

A 32-bit unsigned integer reserved for future use or to indicate additional information in specific packet types. In some cases, like in QUERY messages, it might be used to specify the type of metadata being queried. In other cases, it might simply be set to zero.

Field Name	Size	Description
packet_type	8 bits (1 byte)	Identifies the type of packet (e.g., HELLO, QUERY, RESPONSE).
packet_length	8 bits (1 byte)	Specifies the payload size or total header length in certain cases.
transaction_id	16 bits (2 bytes)	Unique identifier for tracking request-response transactions.
reserved	32 bits (4 bytes)	Reserved for future use or additional metadata; often set to zero.

Packet Types:

Type Value	Message Type	Description
0x01	HELLO	Node announces its presence
0x02	QUERY	Request for specific metadata
0x03	RESPONSE	Response to QUERY with metadata

Payload Format: The payload is optional and depends on the message type.

- HELLO (0x01):** No additional payload.
- QUERY (0x02):** No additional payload but reserved bit is used to set ALL_TYPE
- RESPONSE (0x03):** Contains the requested metadata values.

Implementation Details

- The server efficiently utilizes raw sockets (**SOCK_RAW**) to process packets at the network layer.
- The **IP_HDRINCL** option is explicitly set, ensuring control over IP header construction.
- The server properly enables **broadcasting** using **SO_BROADCAST**.
- It allows it to reach multiple clients efficiently.
- The server runs a **dedicated thread (pthread)** for periodic HELLO message broadcasting.
- This **improves responsiveness**, as packet reception & broadcasting do not interfere with each other.
- **Message Type** validation and **Packet Length** validation is enforced.
- The server and the client include **checksum** validation to detect transmission errors. This ensures that received packets are **not corrupted** or **tampered with** during transmission.

Communication Flow

- The server broadcasts HELLO every 10 seconds
- The client begins with user input about which metadata to query
- Client sends a query to active nodes when it receives a HELLO message from that node.
- Server process queries and sends responses immediately
- Client displays the response.
- Client continues in an infinite loop querying metadata at each HELLO from a server.
- Press Ctrl + C to exit.

Limitations

- Currently works only with IPv4
- Each server must be on different IP
- No more than 100 active queries from client side
- Clients at a time can query for a fixed set of metadata (not user-input)
- Transaction ID limits the number of clients.
- BUFFER_SIZE limits the maximum size of a payload.